

M01 - INTRODUÇÃO

Introdução PL/SQL

Linguagem de Programação - Estrutura - Arquitetura - Vantagens

Agenda da Aula

01

O que é PL/SQL

Definição e extensão procedural do SQL Oracle

02

Estrutura do Bloco

DECLARE / BEGIN / EXCEPTION / END

03

Modularidade

Blocos, procedures, funções e packages

04

Arquitetura

Módulo executor: Oracle Server e ferramentas

05

Vantagens

SQL, OO, Portabilidade, Produtividade, Integração

06

Exemplo Prático

Primeiro bloco PL/SQL completo com DBMS_OUTPUT

07

Performance: 1 vs N SQLs

Como PL/SQL reduz o tráfego de rede com um único envio

08

Erros comuns + Exercício

Barras, SERVEROUTPUT, := vs =, END;

O que é PL/SQL?

A PL/SQL é uma linguagem procedural da Oracle que estende a SQL com comandos que permitem a criação de procedimentos de programação.

Com ela, podemos usar comandos de SQL DML para manipular os dados da base de dados Oracle e estabelecer fluxos de controle para processar estes dados.

A linguagem permite a declaração de constantes, variáveis, subprogramas (procedures e funções), que favorecem a estruturação de código, e possui mecanismos para controle de erros de execução. Incorpora os novos conceitos de objeto, encapsulamento e, ainda, permite a interface com rotinas escritas em outras linguagens.

Estrutura do Bloco PL/SQL

A PL/SQL é estruturada em blocos.

Cada bloco pode conter outros blocos.

Em cada um desses blocos, podemos declarar variáveis que deixam de existir quando o bloco termina.

Estrutura do Bloco PL/SQL

```
[ DECLARE
  -- declaração de variáveis, constantes, cursores
]
BEGIN
  -- lógica do bloco (comandos SQL e PL/SQL)
  -- blocos subordinados (embutidos)
[ EXCEPTION
  -- tratamento de erros
]
END;
/
```

Execute em aula: A01_01_sintaxe_bloco.sql

DECLARE (opcional)

Parte declarativa, onde definimos as variáveis locais àquele bloco.

BEGIN (obrigatório)

Parte de lógica, onde definimos a ação que aquele bloco deve realizar, incluindo a declaração de outros blocos subordinados (ou embutidos) a este.

EXCEPTION (opcional)

Parte de tratamento de erros, que permite acesso ao erro ocorrido e à determinação de uma ação de correção.

END (obrigatório)

Finaliza o bloco. No SQL*Plus, a barra (/) na linha seguinte executa o bloco.

Modularidade

Modularidade é um conceito que determina a divisão do programa em módulos com ações bem definidas, que visam a facilitar o entendimento e a manutenção.

A PL/SQL, por possuir uma estrutura de blocos, favorece a modularidade. Além de blocos anônimos, temos a possibilidade de criar procedures e funções armazenadas na base de dados e compartilhadas por outras aplicações, ou packages que permitem que agrupemos procedures, funções e variáveis relacionadas.

Bloco Anônimo

Executado diretamente, sem nome armazenado. Uso pontual no SQL*Plus ou aplicação.

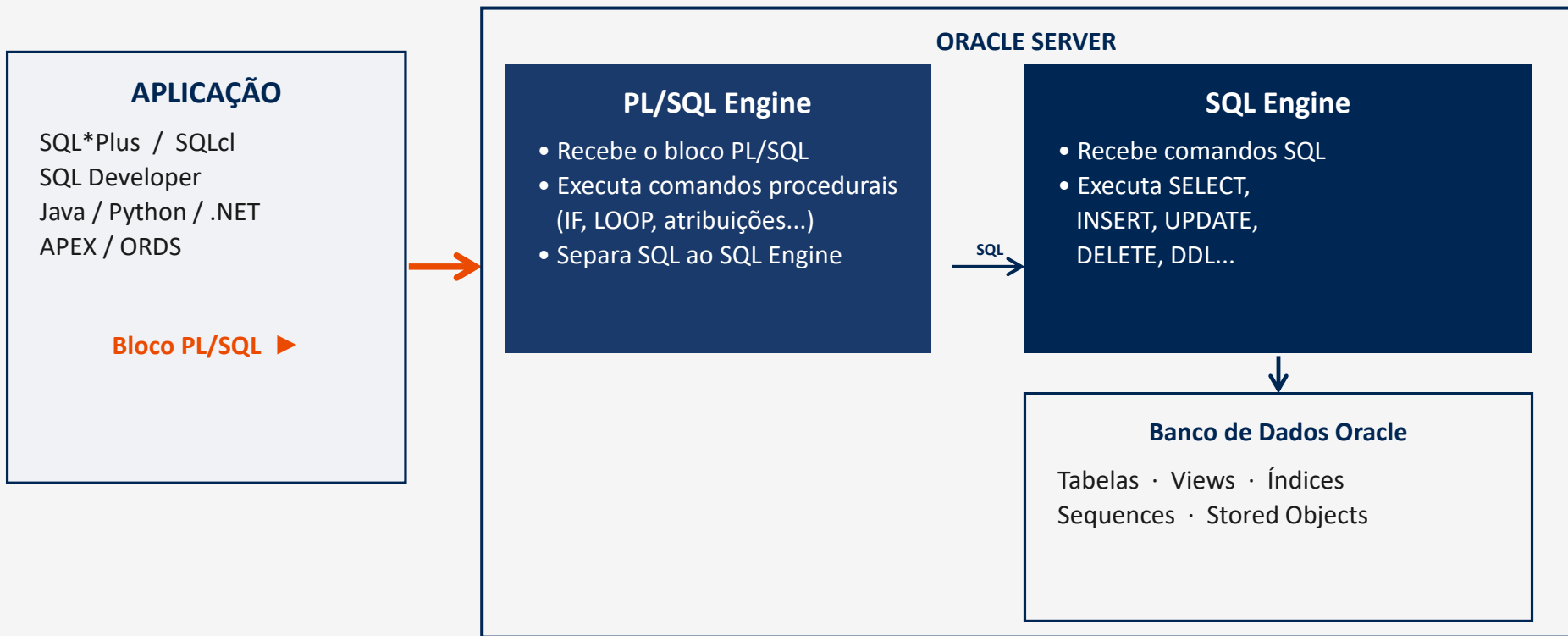
Procedure / Função

Armazenadas na base de dados e compartilhadas por múltiplas aplicações.

Package

Agrupar procedures, funções e variáveis relacionadas em uma única unidade lógica.

Arquitetura PL/SQL



O PL/SQL Engine executa os comandos procedurais localmente e envia apenas os SQL ao SQL Engine — reduzindo round-trips.

Vantagens da PL/SQL

SUPOORTE PARA SQL

A PL/SQL permite a utilização, integrada no código, dos comandos da SQL DML, das funções de SQL, de comandos de controle de cursor e dos comandos para controle da transação (Commit, Rollback, etc.).

PORTABILIDADE

Aplicações escritas em PL/SQL são portáteis para qualquer sistema operacional e plataforma nos quais o Oracle execute. Não há necessidade de customização. Podemos escrever programas ou bibliotecas de programas que podem ser utilizados em ambientes diferentes.

INTEGRAÇÃO COM O ORACLE

Os atributos %TYPE e %ROWTYPE permitem a integração com o dicionário de dados Oracle, pois poderemos declarar uma variável com o mesmo tipo de uma coluna definida em uma tabela do banco de dados.

SUPOORTE PARA PROGRAMAÇÃO ORIENTADA A OBJETO

Quando criamos um tipo objeto, definimos suas características através de seus atributos e métodos. Os métodos são escritos em PL/SQL. Podemos declarar variáveis com qualquer dos tipos predefinidos existentes no banco de dados ou com os tipos criados pelo usuário, inclusive tipos objeto.

PRODUTIVIDADE

O aprendizado da PL/SQL pode ser aproveitado no desenvolvimento de aplicações batch, online, relatórios, APIs e integrações. Uma vez dominada a linguagem, a lógica de programação se aplica a qualquer ferramenta do ecossistema Oracle.

Exemplo Prático: Primeiro Bloco PL/SQL

Bloco anônimo básico: estrutura DECLARE / BEGIN / EXCEPTION / END com DBMS_OUTPUT

SQL*Plus / SQL Developer

```
DECLARE
    v_mensagem    VARCHAR2(100) := 'Primeiro bloco PL/SQL';
    v_numero       NUMBER       := 2025;
BEGIN
    DBMS_OUTPUT.PUT_LINE('Mensagem: ' || v_mensagem);
    DBMS_OUTPUT.PUT_LINE('Ano: '      || v_numero);
EXCEPTION
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('Erro: ' || SQLERRM);
END;
/
```

DECLARE

Seção opcional. Declara variáveis locais ao bloco.

v_mensagem

Variável VARCHAR2 inicializada com :=

BEGIN

Início obrigatório da lógica do bloco.

DBMS_OUTPUT

Package Oracle para exibir texto no console.

EXCEPTION

Captura erros em tempo de execução.

END; /

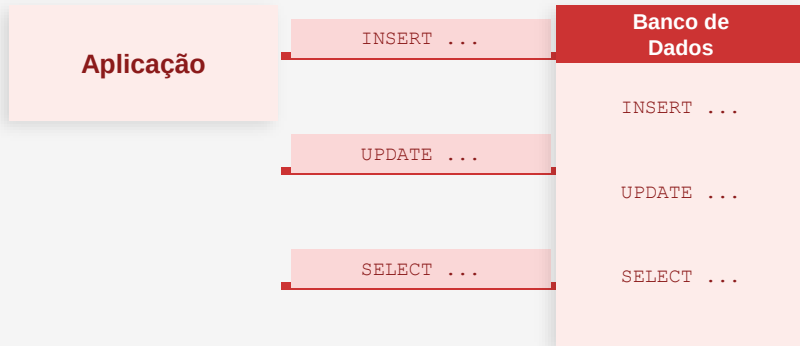
END encerra o bloco. A barra (/) executa no SQL*Plus.

Execute em aula: A01_07_primeiro_bloco_completo.sql

Performance: Reduzindo o Tráfego de Rede

A utilização da PL/SQL pode reduzir o tráfego na rede pelo envio de um bloco contendo diversos comandos de SQL agrupados em blocos para o Oracle.

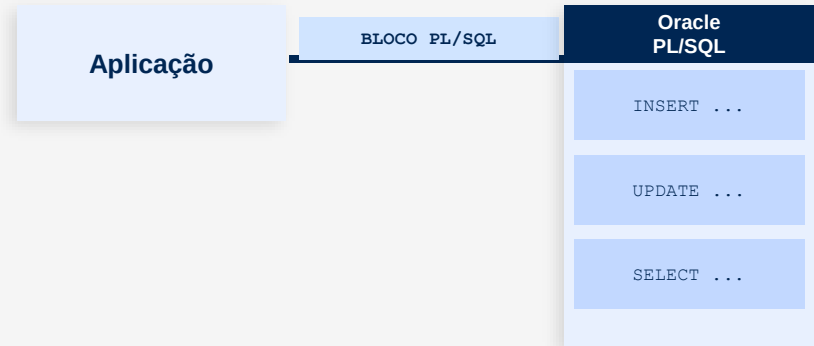
SEM PL/SQL: N Round Trips



3 comandos enviados separadamente
3 round trips de rede → maior latência

VS

COM PL/SQL: 1 Round Trip



1 bloco enviado ao Oracle
PL/SQL processa os SQLs internamente → menos tráfego

Execute em aula: A01_08_performance_1bloco.sql

Variáveis de Substituição (&)

Recurso do SQL*Plus — o valor é digitado pelo usuário ao executar e substituído antes de chegar ao Oracle.

&variavel

Solicita o valor toda vez que aparece no bloco.

```
VARCHAR2 := '&variavel'   NUMBER := &variavel
```

&&variavel

Solicita o valor uma só vez e reutiliza no mesmo bloco.

Útil quando a variável aparece mais de uma vez.

ACCEPT var TYPE tipo PROMPT 'texto'

Prompt customizado com tipo e valor padrão.

Controla melhor a entrada antes do bloco executar.

Regra de aspas:

`VARCHAR2` → `v_nome := '&v_nome'`

`NUMBER` → `v_sal := &v_sal`

(sem aspas para números)

Execute em aula: A01_08_performance_1bloco_USER_SCOTT.sql

EXEMPLO — os 3 recursos em ação

```
-- 1. ACCEPT: prompt antes do bloco
ACCEPT v_nome CHAR    PROMPT 'Seu nome : '
ACCEPT v_sal  NUMBER  PROMPT 'Salario   : '

-- 2. Bloco com & e &&
DECLARE
    v_nome    VARCHAR2(50) := '&v_nome';
    v_sal     NUMBER       := &v_sal;
    v_bonus   NUMBER       := v_sal * 0.10;
BEGIN
    DBMS_OUTPUT.PUT_LINE('Nome   : ' || v_nome);
    DBMS_OUTPUT.PUT_LINE('Sal    : ' || v_sal);
    DBMS_OUTPUT.PUT_LINE('Bonus  : ' || v_bonus);
END;
/
```

Saída (usuário digitou 'Ana' e 5000):

```
Nome   : Ana
Sal    : 5000
Bonus  : 500
```

Hora de Praticar!

Abra o SQL Developer ou SQL*Plus e execute cada script abaixo.

```
SET SERVEROUTPUT ON  -- cole isso antes de executar qualquer script
```

Execute em aula: A01_01_sintaxe_bloco.sql

Veja a estrutura DECLARE/BEGIN/END funcionando pela 1a vez

Execute em aula: A01_03_palavras_reservadas.sql

Descubra quais palavras o PL/SQL reserva para si

Execute em aula: A01_05_literais.sql

Experimente literais numericos, texto e booleanos

Execute em aula: A01_07_primeiro_bloco_completo.sql

Seu primeiro bloco PL/SQL completo com saida na tela

Execute em aula: A01_02_identificadores.sql

Teste nomes validos e invalidos — o compilador vai reclamar!

Execute em aula: A01_04_comentarios.sql

Comente blocos com -- e /* */ e veja o efeito

Execute em aula: A01_06_fim_de_linha.sql

O / no SQL*Plus — sem ele o bloco nao executa!

Execute em aula: A01_08_performance_1bloco.sql

Compare N roundtrips vs 1 bloco: veja a diferenca

Erros Comuns do Iniciante

Esquecer a barra (/) no SQL*Plus

O bloco PL/SQL só é executado quando você digita / em uma linha separada. Sem a barra, o SQL*Plus aguarda mais entrada e o bloco não roda.

```
BEGIN
    DBMS_OUTPUT.PUT_LINE('Ola');
END;
-- sem a barra: bloco não executa
/ -- correto: barra em linha própria
```

SERVEROUTPUT desligado

DBMS_OUTPUT.PUT_LINE não exibe nada se o SERVEROUTPUT estiver OFF. Ative antes de executar o bloco.

```
SET SERVEROUTPUT ON;
BEGIN
    DBMS_OUTPUT.PUT_LINE('Visível
    agora!');
END;
/
```

Usar = em vez de := para atribuição

Em PL/SQL, = é operador de comparação. Para atribuir valor a uma variável, use := (dois pontos e igual).

```
DECLARE
    v_x NUMBER;
BEGIN
    v_x := 10; -- correto
    -- v_x = 10; -- ERRO de compilação
END;
/
```

Exercício

Coloque em prática o que você aprendeu nesta aula

Exercicio: bloco_soma.sql

```
-- Exercício M01
-- Escreva um bloco PL/SQL que:
--
-- 1. Declare duas variáveis numéricas: v_a e v_b
-- 2. Atribua os valores 10 e 20 a elas
-- 3. Calcule a soma e armazene em v_soma
-- 4. Exiba no console: 'Soma: <valor>'
-- 5. Adicione uma seção EXCEPTION com WHEN OTHERS
--
-- Dica: use DBMS_OUTPUT.PUT_LINE para exibir o resultado
-- Dica: lembre-se de SET SERVEROUTPUT ON antes de executar
```

Exercícios completos em: M01_exercios_adicionais/ (10 exercícios + gabaritos)

Revisão do M01

- PL/SQL é uma linguagem procedural Oracle que estende o SQL
- Estrutura em blocos: DECLARE / BEGIN / EXCEPTION / END
- Modularidade: blocos anônimos, procedures, funções e packages
- Arquitetura: módulo executor no Oracle Server e nas ferramentas
- Performance: 1 bloco PL/SQL substitui N comandos SQL individuais na rede
- Vantagens: Suporte SQL e OO, Portabilidade, Produtividade, Integração

Próxima aula: M02 - Fundamentos da Linguagem

Características, Tipos de Dados, Declarações e Escopo